

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:50

Annexure A

DESIGN TASK

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

DESIGN TASK

Q1.Design an unambiguous context-free grammar (CFG) to generate arithmetic expressions consisting of the symbols id, *, -, (), $\uparrow$ (where $\uparrow$ represents exponentiation), ensuring correct operator precedence and associativity.

Further, design a shift-reduce parser for the given grammar and demonstrate its working by parsing the input string:

id - id * id $\uparrow$ id (id $\uparrow$ id) $\uparrow$ id

Your design should:

- (i). Clearly specify production rules that eliminate ambiguity
- (ii).Handle operator precedence ($\uparrow > * > -$) and associativity
- (iii).Show step-by-step shift-reduce parsing actions
- (iv).Indicate reductions and handle parsing decisions properly

S.NO	Ambiguous Grammar	Unambiguous Grammar
1.	In ambiguous grammar, the leftmost and rightmost derivations are not same.	In unambiguous grammar, the leftmost and rightmost derivations are same.
2.	Amount of non-terminals in ambiguous grammar is less than in unambiguous grammar.	Amount of non-terminals in unambiguous grammar is more than in ambiguous grammar.
3.	Length of the parse tree in ambiguous grammar is comparatively short.	Length of the parse tree in unambiguous grammar is comparatively large.
4.	Speed of derivation of a tree in ambiguous grammar is faster than that of unambiguous grammar.	Speed of derivation of a tree in unambiguous grammar is slower than that of ambiguous grammar.
5.	Ambiguous grammar generates more than one parse tree.	Unambiguous grammar generates only one parse tree.
6.	Ambiguous grammar contains ambiguity.	Unambiguous grammar does not contain any ambiguity.

Q2. Design an LL(1) parser for the given grammar:

$D \rightarrow T L ;$

$L \rightarrow L , id \mid id$

$T \rightarrow int \mid float$

Your design should include the following:

- (i). Transform the grammar by eliminating left recursion (if present) and ensure it is suitable for LL(1) parsing.
- (ii). Design and compute FIRST and FOLLOW sets for all non-terminals in the transformed grammar.
- (iii). Construct the LL(1) parsing table based on the computed FIRST and FOLLOW sets.
- (iv). Demonstrate the working of the parser by parsing the input string:

int id, id;

- (a). Show all stack operations clearly
- (b). Indicate each parsing action (match, expand, accept, error if any)
- (c) Finally, design and illustrate the parse tree for the given input

Q3. Design and implement a recursive descent parser for the following grammar:

$S \rightarrow AB \mid BC$

$A \rightarrow a$

$B \rightarrow b$

$C \rightarrow c$

Your design should include:

- (i). Development of procedures for each non-terminal (S, A, B, C)
- (ii). Application of top-down parsing strategy without backtracking (if possible)
- (iii). Proper handling of input string matching using a predictive approach
- (iv). Demonstration of the parser by testing strings such as ab and bc
- (v). Clearly indicating whether the input string is accepted or rejected based on the grammar

Q4.Design an operator precedence parser for arithmetic expressions involving the operators +, *, and parentheses () with operands as id.

Your design should include:

- (i). Construction of an unambiguous grammar suitable for operator precedence parsing
- (ii). Definition of operator precedence relations ($<\cdot$, $=\cdot$, $\cdot>$) among terminals
- (iii). Development of the operator precedence table
- (iv). Implementation of the parsing algorithm using stack-based shift-reduce technique
- (v). Demonstration of the parser by parsing the input string:

id + id * id

- (a). Show all shift and reduce actions clearly
- (b). Indicate how precedence rules are applied during parsing
- (c). Ensure correct handling of operator precedence and associativity

Q5. Design and construct an SLR parser for the following grammar:

$S \rightarrow 0S0 \mid 1S1 \mid 10$

Your design should include:

- (i). Construction of the augmented grammar
- (ii). Development of the canonical LR(0) item sets
- (iii). Computation of FOLLOW sets required for SLR parsing
- (iv). Construction of the SLR parsing table (ACTION and GOTO)
- (v). Demonstration of the parser by parsing the input string: 0100
 - (a). Show all stack contents, input, and parsing actions clearly
 - (b). Indicate shift, reduce, and accept actions step-by-step

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	30	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete logic	Mostly correct construction with minor errors	Partial construction with missing logic	Incorrect or incomplete construction
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	10	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation